DIGITAL DOCUMENT MANAGEMENT SYSTEM

Project Documentation with Node.js, Express, HTML, CSS and JavaScript

Project Type: Web Application

Prepared for: Academic Project / Practical Record

A simple web-based system to upload, organize, search and manage digital documents.

1. Abstract

The Digital Document Management System (DDMS) is a web application designed to store and organize document details digitally. Users can add document records, search for documents, view document information and delete records when they are no longer needed. The sample project uses Node.js and Express for the server, HTML/CSS/JavaScript for the user interface, and a JSON file as simple local storage. This is an educational prototype; a production system should use a database, authentication, access permissions, backups and secure file storage.

2. Theme

The theme of this project is **paperless document management**. It reduces manual record keeping by helping an organization maintain searchable digital document records in one place.

Objectives

• Store document details in a structured digital format.

• Make it easier to search by title, category or filename.

• Reduce time spent locating records.

• Provide a simple and user-friendly interface.

• Demonstrate a basic full-stack application using Node.js.

Scope

The prototype manages document metadata and uploaded files in a local uploads folder. It is suitable for learning and demonstration, not for confidential or high-volume business use without additional security and database work.

3. Technologies Used

Technology	Purpose
HTML	Creates page structure and forms.
CSS	Styles the page and document list.
JavaScript	Handles form submission, search and page updates.
Node.js	Runs server-side JavaScript.
Express.js	Defines routes and handles HTTP requests.
Multer	Accepts file uploads.
JSON file	Stores document metadata for this small demo.

4. Main Modules

• **Add Document:** Upload a file and enter its title and category.

• **View Documents:** Display saved document records.

• **Search:** Filter records by title, category or filename.

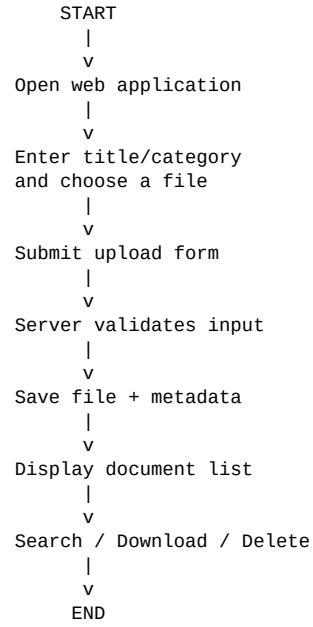
• **Download:** Open or download a stored file using its file link.

• **Delete:** Remove a record and its associated file.

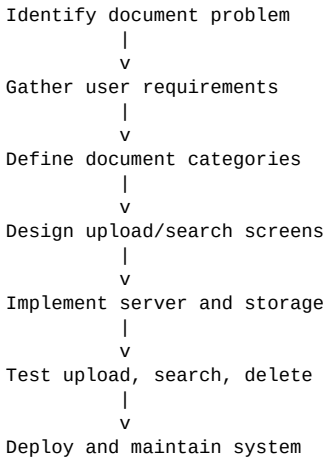
5. Workflow Explanation

The user opens the web page and fills in the document title, category and file. The browser sends the form to the Express server. Multer saves the file in the uploads folder. The server records the title, category, filename and date in a JSON file. The page then refreshes its document list. Search filters the visible records, while delete removes the selected record.

Workflow Flowchart



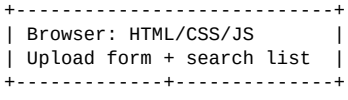
6. Business Analysis Flowchart

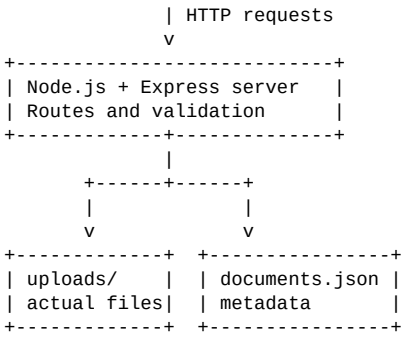


Business Need

Manual document filing can lead to duplicate records, slow retrieval and misplaced files. A digital index improves discoverability and provides a consistent workflow. For an actual organization, business rules should specify who may upload, read, edit and delete each document.

7. System Architecture





8. Algorithm Design

Upload Document Algorithm

1. Start.
2. Receive title, category and selected file.
3. Check that title, category and file are present.
4. Save file to the uploads folder.
5. Create a unique document ID.
6. Add document metadata to documents.json.
7. Return success response.
8. End.

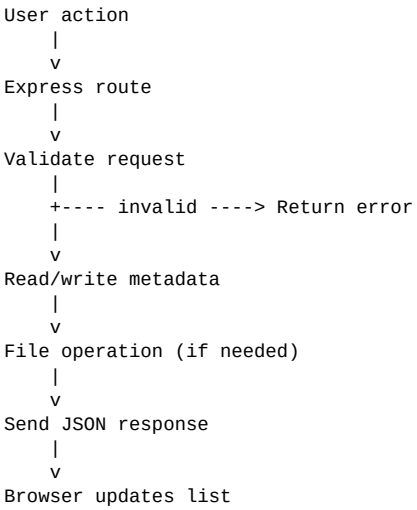
Search Algorithm

1. Read all document records.
2. Convert search text to lowercase.
3. Compare it with title, category and filename.
4. Display records containing the search text.
5. If no records match, show an empty result.

Delete Algorithm

1. Receive document ID.
2. Find the matching record.
3. If no record exists, return 404.
4. Remove its file if it exists.
5. Remove the metadata record.
6. Save updated JSON file and return success.

9. Node.js Business Logic Diagram



10. Project Tree Diagram

```
digital-document-management/
|-- package.json
|-- server.js
|-- documents.json       (created automatically)
|-- uploads/             (created automatically)
`-- public/
```

```
    |-- index.html
    |-- style.css
    `-- app.js
```

11. Implementation Code

Create the folders and files exactly as shown in the project tree. The code below is a learning prototype. Use Node.js 18 or later.

11.1 package.json

```
{
  "name": "digital-document-management",
  "version": "1.0.0",
  "description": "Educational Digital Document Management System",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.21.2",
    "multer": "^1.4.5-lts.1"
  }
}
```

11.2 server.js

```
const express = require("express");
const multer = require("multer");
const fs = require("fs");
const path = require("path");
const crypto = require("crypto");

const app = express();
const PORT = process.env.PORT || 3000;
const ROOT = __dirname;
const PUBLIC_DIR = path.join(ROOT, "public");
const UPLOAD_DIR = path.join(ROOT, "uploads");
const DATA_FILE = path.join(ROOT, "documents.json");

fs.mkdirSync(UPLOAD_DIR, { recursive: true });
if (!fs.existsSync(DATA_FILE)) fs.writeFileSync(DATA_FILE, "[]");

app.use(express.static(PUBLIC_DIR));
app.use("/uploads", express.static(UPLOAD_DIR));

function readDocuments() {
  try { return JSON.parse(fs.readFileSync(DATA_FILE, "utf8")); }
  catch (err) { return []; }
}
function writeDocuments(items) {
  fs.writeFileSync(DATA_FILE, JSON.stringify(items, null, 2));
}

const storage = multer.diskStorage({
  destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
  filename: (_req, file, cb) => {
    const safeOriginal = path.basename(file.originalname)
      .replace(/[^a-zA-Z0-9._-]/g, "_");
    cb(null, `${Date.now()}-${crypto.randomUUID()}-${safeOriginal}`);
  }
});

const allowedExtensions = new Set([
  ".pdf", ".doc", ".docx", ".txt", ".png", ".jpg", ".jpeg"
]);
const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (_req, file, cb) => {
    const ext = path.extname(file.originalname).toLowerCase();
    if (allowedExtensions.has(ext)) return cb(null, true);
    cb(new Error("Unsupported file type. Use PDF, DOC, DOCX, TXT, PNG or JPG."));
  }
});

app.get("/api/documents", (_req, res) => {
  const q = String(_req.query.q || "").toLowerCase().trim();
```

```
  const docs = readDocuments().filter(d =>
    [d.title, d.category, d.originalName].some(v =>
      String(v || "").toLowerCase().includes(q)
    )
  );
  res.json(docs);
});

app.post("/api/documents", (req, res) => {
  upload.single("document")(req, res, err => {
    if (err) return res.status(400).json({ error: err.message });
    const title = String(req.body.title || "").trim();
    const category = String(req.body.category || "").trim();

    if (!title || !category || !req.file) {
      if (req.file) fs.unlinkSync(req.file.path);
      return res.status(400).json({
        error: "Title, category and a document file are required."
      });
    }

    const docs = readDocuments();
    const doc = {
      id: crypto.randomUUID(),
      title,
      category,
      originalName: req.file.originalname,
      storedName: req.file.filename,
      uploadedAt: new Date().toISOString()
    };
    docs.unshift(doc);
    writeDocuments(docs);
    res.status(201).json(doc);
  });
});

app.delete("/api/documents/:id", (req, res) => {
  const docs = readDocuments();
  const doc = docs.find(d => d.id === req.params.id);
  if (!doc) return res.status(404).json({ error: "Document not found." });

  const filePath = path.join(UPLOAD_DIR, doc.storedName);
  if (fs.existsSync(filePath)) fs.unlinkSync(filePath);
  writeDocuments(docs.filter(d => d.id !== doc.id));
  res.json({ message: "Document deleted." });
});

app.use((err, _req, res, _next) => {
  console.error(err);
  res.status(500).json({ error: "Unexpected server error." });
});

app.listen(PORT, () => {
  console.log(`DDMS running at http://localhost:${PORT}`);
});
```

11.3 public/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Digital Document Management System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="container">
    <header>
      <h1>Digital Document Management System</h1>
      <p>Upload, organize and find your documents in one place.</p>
    </header>

    <section class="card">
      <h2>Add a document</h2>
      <form id="uploadForm">
        <label>Document title
          <input name="title" required maxlength="120" placeholder="e.g. Project Report">
        </label>
        <label>Category
          <select name="category" required>
```

```
            <option value="">Choose category</option>
            <option>Academic</option>
            <option>Office</option>
            <option>Personal</option>
            <option>Other</option>
          </select>
        </label>
        <label>Select file (maximum 5 MB)
          <input name="document" type="file"
            accept=".pdf,.doc,.docx,.txt,.png,.jpg,.jpeg" required>
        </label>
        <button type="submit">Upload Document</button>
        <p id="message" role="status"></p>
      </form>
    </section>

    <section class="card">
      <h2>My documents</h2>
      <input id="search" type="search" placeholder="Search title, category or filename">
      <div id="documentList" aria-live="polite"></div>
    </section>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

11.4 public/style.css

```
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f1f5f9;
  color: #172033;
}
.container { max-width: 900px; margin: 30px auto; padding: 0 16px; }
header { text-align: center; margin-bottom: 24px; }
h1 { color: #163a5f; }
.card {
  background: white; padding: 22px; margin: 18px 0;
  border-radius: 12px; box-shadow: 0 3px 12px #00000012;
}
label { display: block; margin: 12px 0; font-weight: bold; }
input, select {
  width: 100%; padding: 11px; margin-top: 6px;
  border: 1px solid #bcc7d3; border-radius: 7px;
}
button {
  border: 0; border-radius: 7px; padding: 11px 16px;
  background: #163a5f; color: white; cursor: pointer;
}
button:hover { opacity: .9; }
.doc {
  border: 1px solid #d7e0e8; border-radius: 8px;
  padding: 14px; margin-top: 12px;
}
.doc h3 { margin: 0 0 6px; }
.doc a { margin-right: 14px; }
.delete { background: #a12b2b; }
#message { min-height: 20px; }
```

11.5 public/app.js

```
const form = document.getElementById("uploadForm");
const list = document.getElementById("documentList");
const search = document.getElementById("search");
const message = document.getElementById("message");

async function loadDocuments() {
  const response = await fetch("/api/documents?q=" + encodeURIComponent(search.value));
  const docs = await response.json();
  list.replaceChildren();

  if (!docs.length) {
    list.textContent = "No documents found.";
    return;
  }

  docs.forEach(doc => {
    const card = document.createElement("article");
```

```
    card.className = "doc";

    const title = document.createElement("h3");
    title.textContent = doc.title;
    const info = document.createElement("p");
    info.textContent = `${doc.category} • ${doc.originalName} • ` +
      new Date(doc.uploadedAt).toLocaleString();

    const link = document.createElement("a");
    link.href = "/uploads/" + encodeURIComponent(doc.storedName);
    link.textContent = "Open / Download";
    link.target = "_blank";
    link.rel = "noopener";

    const remove = document.createElement("button");
    remove.className = "delete";
    remove.textContent = "Delete";
    remove.addEventListener("click", async () => {
      if (!confirm("Delete this document?")) return;
      const r = await fetch("/api/documents/" + encodeURIComponent(doc.id),
        { method: "DELETE" });
      if (r.ok) loadDocuments();
      else alert("Could not delete document.");
    });

    card.append(title, info, link, remove);
    list.append(card);
  });
}

form.addEventListener("submit", async event => {
  event.preventDefault();
  message.textContent = "Uploading...";
  const response = await fetch("/api/documents", {
    method: "POST",
    body: new FormData(form)
  });
  const result = await response.json();

  if (!response.ok) {
    message.textContent = result.error || "Upload failed.";
    return;
  }

  message.textContent = "Document uploaded successfully.";
  form.reset();
  await loadDocuments();
});

search.addEventListener("input", loadDocuments);
loadDocuments();
```

12. Code Explanation

• **Express:** creates the server and API routes.

• **Multer:** receives uploaded files and stores them in uploads/.

• **readDocuments() / writeDocuments():** read and update metadata in documents.json.

• **GET /api/documents:** returns document records and supports keyword search.

• **POST /api/documents:** validates fields and saves a new record.

• **DELETE /api/documents/:id:** removes the file and its metadata.

• **app.js:** sends requests to the server and displays returned records.

• **style.css:** controls page layout, colours, spacing and buttons.

13. Input and Output

Input

• Document title: Project Report

• Category: Academic

• File: project-report.pdf

Expected Output

After a successful upload, the page shows a document card containing the title, category, original filename, upload date/time, an Open / Download link and a Delete button.

Example Output Record

```
{
  "id": "generated-unique-id",
  "title": "Project Report",
  "category": "Academic",
  "originalName": "project-report.pdf",
  "storedName": "generated-file-name.pdf",
  "uploadedAt": "2026-10-14T10:00:00.000Z"
}
```

14. How to Run the Project

```
1. Install Node.js (version 18 or later).
2. Create a folder named digital-document-management.
3. Create the files and folders shown in the project tree.
4. Open a terminal inside the project folder.
5. Run: npm install
6. Run: npm start
7. Open: http://localhost:3000
8. Upload a small test PDF or image and try search/delete.
```

Note: Do not type the URL into the terminal. Start the server first, then open the address in a browser. Keep the terminal window running while testing.

15. Screenshots / Pictures to Add

Take actual screenshots after running the project. The following are the recommended screenshots for your report:

• **Screenshot 1 – Home page:** the page title and Add a document form.

• **Screenshot 2 – Upload form:** title, category and selected filename before upload.

• **Screenshot 3 – Document list:** a successful upload displayed with Open / Download and Delete.

• **Screenshot 4 – Search result:** search field with filtered documents.

• **Screenshot 5 – Empty state:** a search term that returns “No documents found.”

Screenshots should be captured from your own running application so they accurately show your implementation. On Windows, press Win + Shift + S to capture a selected area; on many laptops, use the Snipping Tool.

16. Testing

Test	Action	Expected result
1	Upload valid PDF with all fields	Success message and document card
2	Submit without title/category/file	Validation error
3	Search by title	Matching record is shown

4	Search a non-existing name	No documents found
5	Delete a document	Record and stored file are removed
6	Upload unsupported extension	Upload is rejected

17. Limitations and Future Enhancements

• JSON storage is suitable only for a small demonstration and is not designed for simultaneous multi-user production use.

• The prototype has no login system or role-based access control.

• Uploaded files are served as static files; do not use this setup for private or confidential documents.

• Future versions can use MongoDB or PostgreSQL, user authentication, permissions, audit logs, file versioning, backups, encryption, cloud storage and OCR-based text search.

18. Conclusion

The Digital Document Management System demonstrates how a basic paperless document workflow can be implemented with web technologies. It allows users to upload document files, store metadata, search records and delete documents through a simple interface. The project meets its learning objectives and can be extended with a database, authentication and stronger security for real-world use.

19. References

• Node.js Documentation – https://nodejs.org/docs/latest/api/

• Express.js Documentation – https://expressjs.com/

• Multer Documentation – https://github.com/expressjs/multer

• MDN Web Docs – HTML, CSS, JavaScript and Fetch API: https://developer.mozilla.org/

• OWASP File Upload Cheat Sheet – https://cheatsheetseries.owasp.org/

Educational use note: Test only with files you are allowed to upload. Before real deployment, add authentication, authorization, robust validation, malware scanning, secure storage and backups.